

SKYLOCK CYBER DEFENSE

Web Application Penetration Test Report

OWASP Juice Shop — Full Vulnerability Assessment

OPTION 1



SKYLOCK CYBER DEFENSE WEB APPLICATION SECURITY LAB - PART 2

STRENGTHENING LEGACY SYSTEMS THROUGH PENTESTING

OPTION 2

**SKYLOCK
CYBER DEFENSE**



DVWA: XSS ADVANCED TESTING (MEDIUM/IMPOSSIBLE)

ANALYZING MITIGATIONS AND SECURE CODING

OPTION 3



SKYLOCK CYBER DEFENSE

XSS MITIGATION ANALYSIS: BEYOND LOW SECURITY
stored & reflected, medium, impossible, source code analysis



Client	OWASP Juice Shop (Lab Environment)
Prepared By	Brian Muema
Assessment Type	Web Application Penetration Test — OWASP Juice Shop
Target URL	http://localhost:3000
Environment	Kali Linux VM, Docker, Firefox, Burp Suite Community Edition
Vulnerabilities Tested	Reflected XSS, Stored XSS, SQL Injection, IDOR, Token Exposure
Classification	CONFIDENTIAL

1. Engagement Summary	3
2. Environment Setup	3
2.1 Docker Container Deployment	3
2.2 Container Shutdown Verification	4
3. Reconnaissance and Application Mapping	4
4. Vulnerability Findings	5
4.1 Reflected Cross-Site Scripting (Search Bar)	5
4.1.1 Vulnerability Description	5
4.1.2 Exploitation	5
4.1.3 Impact	6
4.1.4 Remediation	7
4.2 Stored Cross-Site Scripting (Product Reviews)	7
4.2.1 Vulnerability Description	7
4.2.2 Exploitation	7
4.2.3 Impact	8
4.2.4 Remediation	8
4.3 SQL Injection — Authentication Bypass	8
4.3.1 Vulnerability Description	9
4.3.2 Exploitation	9
4.3.3 Impact	10
4.3.4 Remediation	10
4.4 Insecure Direct Object Reference — Administration Panel	10
4.4.1 Vulnerability Description	10
4.4.2 Exploitation	10
4.4.3 Impact	11
4.4.4 Remediation	11
4.5 Session Token and JWT Exposure via Traffic Interception	11
4.5.1 Vulnerability Description	11
4.5.2 Exploitation	12
4.5.3 Impact	12
4.5.4 Remediation	12
5. Findings Summary	13
6. Mitigation Analysis	13
6.1 Input Validation vs. Output Encoding	13
6.2 Parameterized Queries	13
6.3 Server-Side Authorization	13
6.4 Defense in Depth	14
7. Reflection	14
7.1 Key Observations	14
7.2 Patterns for Future Assessments	14
8. References	15

1. Engagement Summary

SkyLock Cyber Defense conducted a web application penetration test against the OWASP Juice Shop, a deliberately vulnerable e-commerce application deployed via Docker on a Kali Linux virtual machine. The engagement objective was to identify and demonstrate exploitable vulnerabilities across multiple OWASP Top Ten categories, document evidence of successful exploitation, and provide remediation guidance.

The Juice Shop was launched as a Docker container and verified operational before testing commenced. The assessment covered five primary attack surfaces: reflected and stored Cross-Site Scripting (XSS), SQL injection via the authentication form, Insecure Direct Object Reference (IDOR) through administrative endpoint exposure, and session token capture via Burp Suite proxy interception.

All five attack categories were successfully exploited within the test environment. SQL injection bypassed administrator authentication entirely. Stored XSS payloads were persisted in product reviews and rendered unsanitized. Reflected XSS was partially injected into the search interface. The administration panel was accessed without privilege escalation through direct URL manipulation. Session tokens were captured in plaintext via HTTP request interception.

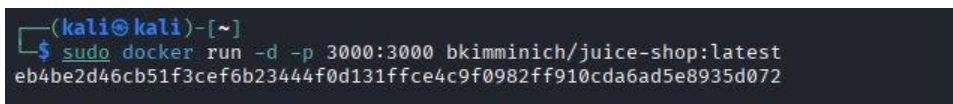
2. Environment Setup

2.1 Docker Container Deployment

The OWASP Juice Shop was deployed using Docker on a Kali Linux virtual machine. The following command was used to pull and run the latest image, binding the application to port 3000 on the host:

```
sudo docker run -d -p 3000:3000 bkimminich/juice-shop:latest
```

Screenshot 1 below shows the Docker run command executed in the terminal, with the full container SHA-256 hash confirming successful image pull and container instantiation.

A terminal window with a dark background. The prompt is (kali@kali)-[~]. The command entered is \$ sudo docker run -d -p 3000:3000 bkimminich/juice-shop:latest. The output is eb4be2d46cb51f3cef6b23444f0d131ffce4c9f0982ff910cda6ad5e8935d072.

```
(kali@kali)-[~]  
$ sudo docker run -d -p 3000:3000 bkimminich/juice-shop:latest  
eb4be2d46cb51f3cef6b23444f0d131ffce4c9f0982ff910cda6ad5e8935d072
```

Container status was verified using the docker ps command, confirming the Juice Shop container was active and port 3000 was correctly mapped:

```
(kali㉿kali)-[~]
$ sudo docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATE
eb4be2d46cb5   bkimminich/juice-shop:latest       "/nodejs/bin/node /j... 19 sec
cp, [::]:3000→3000/tcp   cool_hamilton
```

After confirming the container was running, the application was accessed via Firefox at <http://localhost:3000>. The Juice Shop landing page loaded successfully, confirming the test environment was operational and ready for exploitation testing.

2.2 Container Shutdown Verification

Following completion of all test activities, the Docker container was stopped and the `ps` command was used to confirm no containers remained active. Screenshot 7 below shows the empty container listing after shutdown, confirming the test environment was cleanly terminated.

```
(kali㉿kali)-[~]
$ sudo docker ps
[sudo] password for kali:
CONTAINER ID   IMAGE    COMMAND                  CREATED   STATUS    PORTS   NAMES
(kali㉿kali)-[~]
$
```

3. Reconnaissance and Application Mapping

Reconnaissance was conducted through manual browsing of the Juice Shop application and passive traffic analysis using Burp Suite. The application presents a standard e-commerce interface with product listings, a search bar, user registration and login forms, product reviews, and an administrative panel accessible via direct URL navigation.

The following attack surfaces were identified during the reconnaissance phase:

- Search bar — accepts user input via query parameter, reflects input in page heading and URL
- Product review form — accepts free-text input, stores and renders content without visible sanitization
- Login form — username and password fields with no visible rate limiting or lockout mechanism
- Administration panel — accessible at `/administration` without authentication enforcement in the client
- HTTP request traffic — session cookies and JWT tokens transmitted in request headers

Burp Suite was configured as an intercepting proxy throughout the engagement, capturing all HTTP and WebSocket traffic between the browser and the Juice Shop backend. This allowed inspection of request parameters, authentication tokens, and response payloads.

4. Vulnerability Findings

4.1 Reflected Cross-Site Scripting (Search Bar)

4.1.1 Vulnerability Description

Reflected XSS occurs when user-supplied input is echoed back in the HTTP response without adequate encoding. The Juice Shop search bar accepts a query string parameter and reflects the input in the page heading and URL bar. An attacker who crafts a malicious URL containing JavaScript can cause the victim's browser to execute arbitrary script when the URL is visited.

OWASP Classification: A03:2021 — Injection / Cross-Site Scripting.

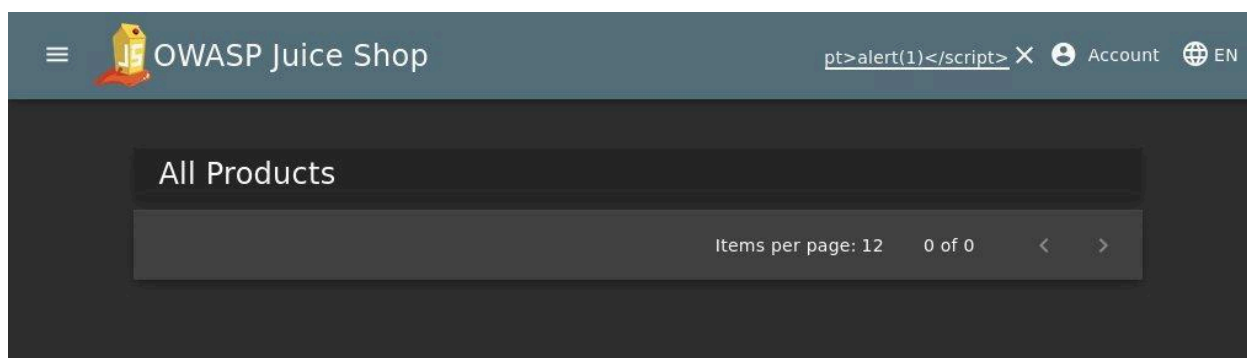
4.1.2 Exploitation

The following payload was injected into the Juice Shop search bar to test for reflected XSS:

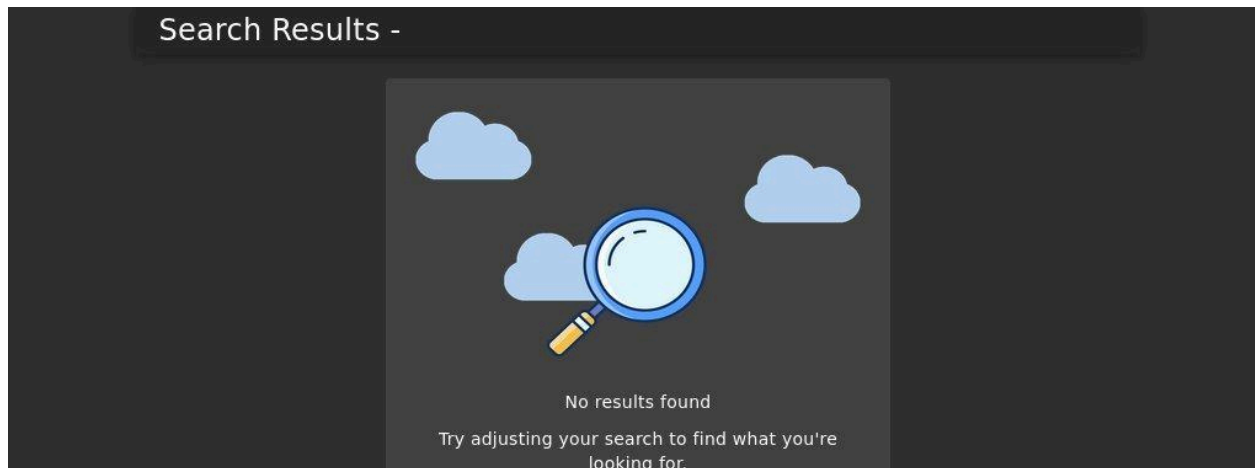
```
"><script>alert(1)</script>
```

The search bar reflected the partial payload in the page heading ('Search Results - ">') and in the browser's navigation bar. The double-quote and greater-than characters from the payload appeared in the heading, demonstrating that the input is interpolated into the DOM without full encoding.

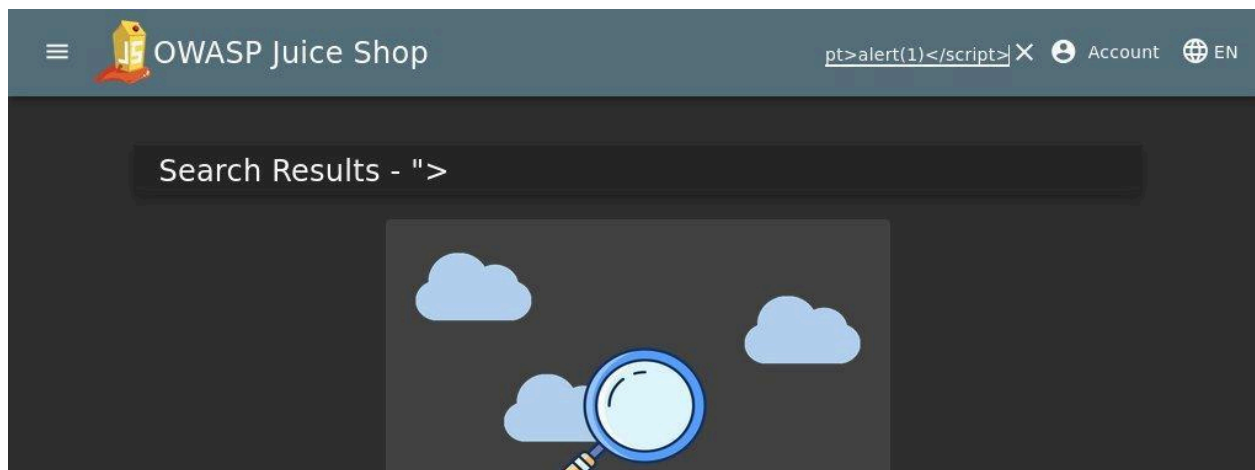
Screenshot 3 shows the search bar displaying the injected payload in the navigation bar while the All Products page renders with zero results:



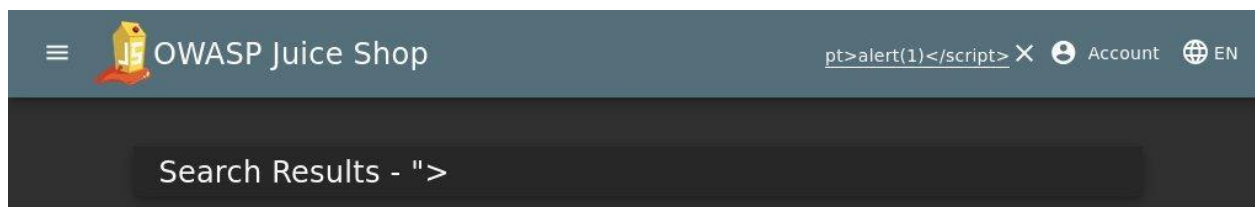
Screenshot 4 shows the Search Results page with no items returned, confirming the payload broke the query structure:



Screenshot 5 shows the heading 'Search Results - ">' confirming the injection characters were reflected in the DOM:



Screenshot 6 provides a closer view of the reflected heading output:



4.1.3 Impact

A fully executed reflected XSS payload would allow an attacker to steal session cookies, redirect users to phishing pages, execute keyloggers, or perform actions on behalf of authenticated users. The partial reflection observed confirms the injection point is active; a refined payload or encoding bypass would likely achieve full script execution.

4.1.4 Remediation

- Apply context-aware output encoding to all search query reflections using a library such as OWASP Java Encoder or DOMPurify.
- Implement a Content Security Policy (CSP) header to restrict inline script execution.
- Validate and sanitize all user-supplied GET parameters on the server side before interpolation into HTML responses.

4.2 Stored Cross-Site Scripting (Product Reviews)

4.2.1 Vulnerability Description

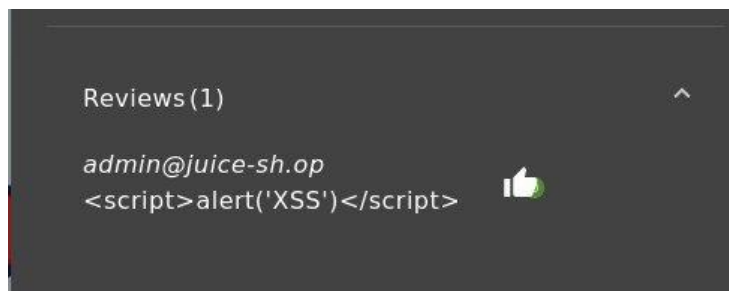
Stored (persistent) XSS occurs when malicious input is saved to the server's database and subsequently rendered in the browser without encoding. Unlike reflected XSS, the payload does not need to be delivered via a crafted URL — it executes automatically for every user who loads the affected page. This makes stored XSS the highest-severity variant of the vulnerability class.

4.2.2 Exploitation

The product review form was identified as accepting free-text input with no client-side validation. The following payload was submitted as a product review:

```
<script>alert('XSS')</script>
```

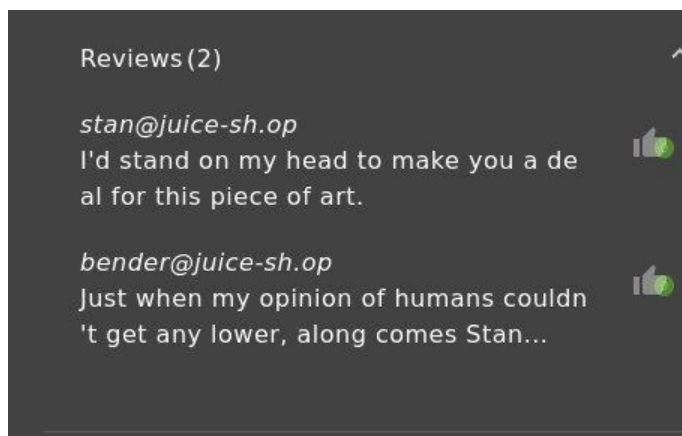
The payload was successfully stored in the Juice Shop database and was observed in the rendered review section. Screenshot 12 shows the stored payload visible in the admin@juice-sh.op review field, rendered as raw HTML text rather than being encoded:



Prior to submitting the XSS payload, the review section was populated with legitimate user reviews. Screenshot 8 shows the initial review state with a single review from admin@juice-sh.op, confirming the review feature was active:



Screenshot 9 shows the product review section with two additional user reviews (stan@juice-sh.op and bender@juice-sh.op), confirming that review data is loaded from the database and rendered in the user's browser without sanitization:



4.2.3 Impact

A stored XSS payload in a product review executes for every authenticated and unauthenticated user who views the affected product page. This enables mass session hijacking, credential theft via form injection, and persistent defacement of the application. In a production environment, this would constitute a critical severity finding.

4.2.4 Remediation

- Encode all user-supplied content before rendering it in HTML contexts using `htmlspecialchars()` or equivalent server-side encoding.
- Implement input validation to reject HTML markup in review fields.
- Apply a strict Content Security Policy to block inline script execution.
- Sanitize stored content at retrieval time using a server-side HTML sanitization library.

4.3 SQL Injection — Authentication Bypass

4.3.1 Vulnerability Description

SQL injection occurs when user-supplied input is incorporated into a database query without parameterization, allowing an attacker to modify the query's logic. In the Juice Shop login form, the email parameter is vulnerable to SQL injection, enabling an attacker to bypass authentication entirely and log in as any user — including the administrator — without knowing the account password.

OWASP Classification: A03:2021 — Injection / SQL Injection.

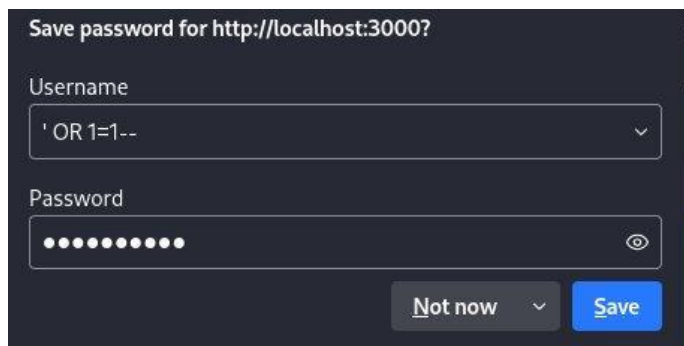
4.3.2 Exploitation

The following payload was entered into the email field of the Juice Shop login form:

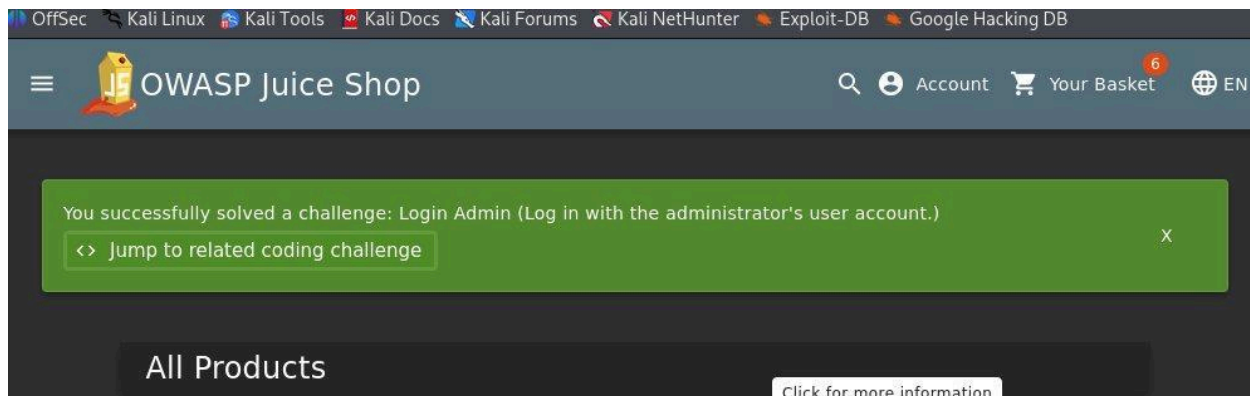
```
' OR 1=1--
```

This payload terminates the SQL string literal with a single quote, appends a condition that always evaluates to true ($1=1$), and comments out the remainder of the query with the double-dash sequence. The resulting query returns the first user record in the database — the administrator account — and grants full authenticated access.

Screenshot 10 shows the browser's password save dialog confirming the login succeeded with the SQL injection payload as the username:



Screenshot 11 shows the Juice Shop homepage after successful authentication, with the green challenge success banner confirming: 'You successfully solved a challenge: Login Admin (Log in with the administrator's user account.):'



4.3.3 Impact

SQL injection in the authentication mechanism represents a critical vulnerability. An attacker can gain administrator-level access to the application without any credentials, allowing them to read all user data, modify prices and product listings, access the administration panel, and potentially pivot to the underlying database server.

4.3.4 Remediation

- Replace dynamic SQL string concatenation with parameterized queries or prepared statements in all database interactions.
- Apply an ORM (Object-Relational Mapping) framework to abstract query construction.
- Implement input validation to reject inputs containing SQL metacharacters.
- Use a Web Application Firewall (WAF) as a defense-in-depth measure to detect and block SQLi patterns.

4.4 Insecure Direct Object Reference — Administration Panel

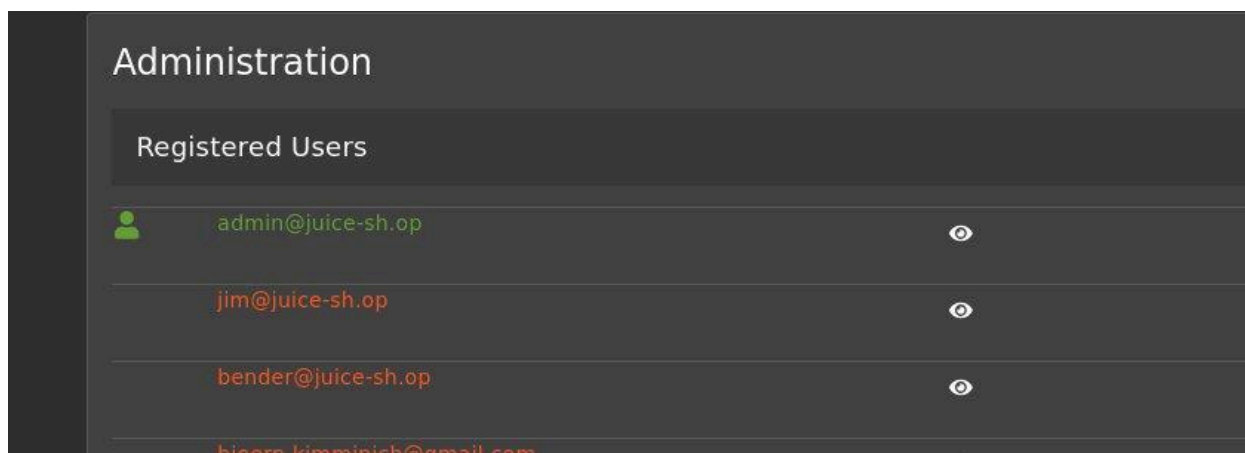
4.4.1 Vulnerability Description

Insecure Direct Object Reference (IDOR) occurs when an application exposes internal implementation objects — such as file paths, database IDs, or administrative endpoints — without enforcing authorization checks. In the Juice Shop, the administration panel is accessible via a direct URL (/administration) without server-side authorization verification beyond the presence of an authenticated session.

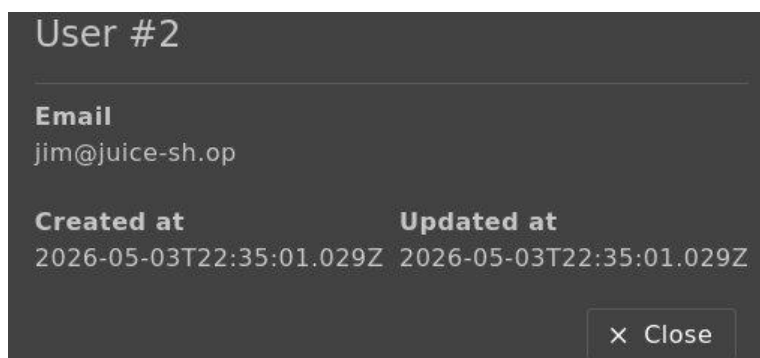
OWASP Classification: A01:2021 — Broken Access Control.

4.4.2 Exploitation

After gaining administrator access via SQL injection (Section 4.3), the /administration route was navigated to directly. The administration panel loaded, displaying a full list of registered users including their email addresses. Screenshot 13 shows the Administration panel with the registered user listing:



Clicking the eye icon next to any user entry revealed full user profile details including email address, user ID, and account timestamps. Screenshot 14 shows the User #2 detail modal, exposing jim@juice-sh.op's account creation and update timestamps:



4.4.3 Impact

Unrestricted access to the administration panel exposes the full registered user database, enabling targeted phishing, credential stuffing attacks against the exposed email accounts, and complete administrative control over the application's data. In a production environment, this would allow an attacker to delete users, modify orders, and alter pricing data.

4.4.4 Remediation

- Enforce server-side role-based access control (RBAC) on all administrative routes.
- Verify the user's role claim on every request to administrative endpoints, not solely at login.
- Remove or obfuscate administrative endpoints from client-side JavaScript bundles to reduce discoverability.
- Log all access attempts to administrative endpoints and alert on anomalous activity.

- Implement a token revocation list or use opaque session tokens stored server-side to enable immediate session invalidation.

5. Findings Summary

The table below consolidates all vulnerabilities identified and exploited during this engagement. Severity ratings are assigned based on exploitability, impact, and CVSS guidance.

Attack	Module	Payload	Outcome	Alert	Notes
Reflected XSS	Search Bar	"><script>alert(1)</script>	Partially reflected	No	Payload visible in URL/heading
Stored XSS	Product Review	<script>alert('XSS')</script>	Stored, rendered raw	Yes*	Payload in review text
SQL Injection	Login Form	' OR 1=1--	Admin login bypassed	N/A	Challenge solved
IDOR	Admin Panel	/administration route	Full user list exposed	N/A	Registered users visible
Token Exposure	WebSocket / Cookie	Burp intercept	JWT/session token captured	N/A	Token visible in request

6. Mitigation Analysis

6.1 Input Validation vs. Output Encoding

The most prevalent class of vulnerability identified in this engagement — XSS in both reflected and stored forms — shares a common root cause: the application processes user input and renders it in HTML contexts without encoding. The correct remediation for XSS is output encoding applied at the point of rendering, not input filtering applied at the point of submission.

Input filtering (blocking or stripping known-bad characters) is inherently fragile because it relies on anticipating every possible malicious pattern. Output encoding is robust because it converts all special characters to their HTML entity equivalents unconditionally, regardless of the input's content. The `htmlspecialchars()` function in PHP, the OWASP Java Encoder, or Angular's built-in template sanitization are examples of correct output encoding approaches.

6.2 Parameterized Queries

The SQL injection finding demonstrates the consequence of dynamic query construction. The correct remediation is parameterized queries (also called prepared statements), where user input is passed as a typed parameter to the query engine rather than concatenated into the

query string. This structurally eliminates the possibility of SQL injection regardless of the input's content.

6.3 Server-Side Authorization

The IDOR finding demonstrates that client-side access control is not access control. Hiding a navigation link or removing a button from the UI does not prevent a technically capable user from navigating to the underlying endpoint directly. Every administrative endpoint must enforce authorization server-side on every request, verifying the authenticated user's role before returning any sensitive data.

6.4 Defense in Depth

No single control is sufficient to prevent all attacks. A defense-in-depth posture combines multiple layers: input validation to catch obviously malformed data, parameterized queries to prevent injection, output encoding to prevent XSS, HTTPS to prevent interception, short-lived tokens to limit session hijacking impact, and access control enforcement at every layer. Each control reduces the attack surface independently, so the failure of one does not expose the application to full compromise.

7. Reflection

7.1 Key Observations

This engagement demonstrated that the OWASP Juice Shop's intentionally vulnerable design closely mirrors real-world vulnerability patterns found in production applications. The SQL injection bypass of authentication using ' OR 1=1-- is not a theoretical attack — it is one of the most commonly observed authentication vulnerabilities in legacy web applications that predate widespread adoption of ORM frameworks.

The stored XSS finding in the product review module illustrates the challenge of securing free-text user input. Organizations frequently implement partial filters — blocking <script> tags while leaving event handlers, SVG-based vectors, and other injection points unmitigated. Comprehensive output encoding at the rendering layer is the only reliable mitigation.

Traffic interception using Burp Suite underscored the value of proxy-based testing in web application assessments. Without inspecting raw HTTP traffic, the presence of long-lived JWTs and unprotected session tokens in request headers would not be visible through browser-based testing alone. The Burp Suite proxy is an essential tool for understanding the full request/response lifecycle.

7.2 Patterns for Future Assessments

- Always test authentication forms for SQL injection before proceeding to other modules — authentication bypass grants the broadest access for subsequent testing.

- Check all free-text input fields for both reflected and stored XSS, paying particular attention to fields whose output is displayed to other users (reviews, comments, profile names).
- Use Burp Suite's HTTP History and Intercept to enumerate all API endpoints, identify hidden parameters, and capture authentication tokens early in the assessment.
- Attempt direct URL navigation to administrative routes without first using the application's navigation to identify IDOR and broken access control findings.
- Verify whether tokens use short expiry times and whether logout actually invalidates the server-side session or only removes the client-side cookie.

8. References

- OWASP Top Ten 2021: <https://owasp.org/Top10/>
- OWASP A01:2021 — Broken Access Control: https://owasp.org/Top10/A01_2021-Broken_Access_Control/
- OWASP A03:2021 — Injection: https://owasp.org/Top10/A03_2021-Injection/
- OWASP XSS Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- OWASP SQL Injection Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- OWASP Juice Shop Project: <https://owasp.org/www-project-juice-shop/>
- Burp Suite Community Edition Documentation: <https://portswigger.net/burp/documentation>
- Docker Hub — bkimminich/juice-shop: <https://hub.docker.com/r/bkimminich/juice-shop>